

130. Le score, et le panneau

Une seconde couche, qui ne défile pas avec le décor.

1. Ce qu'il faut comprendre

Le **panneau** est une couche posée par-dessus le fond. Il ne défile pas : c'est exactement ce qu'on veut d'un score pendant que le décor glisse.

`panneau(x, y)` le place et l'allume, `cacherPanneau()` l'ôte sans rien perdre de son contenu.

Deux détails qui se paient cher si on les oublie. Le compteur `depuis` plutôt qu'un multiple de `images()` : le corps de cette boucle écrit dans le panneau, il déborde d'une image, et le multiple serait enjambé — **le score resterait bloqué à zéro sans que rien ne le dise**. Et `nombrePanneau()` n'est appelé **que quand le score change** : le réécrire à chaque tour alourdit la boucle pour rien.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <panneau>` → [60.1. La fonction panneau\(\) — montrer le panneau](#)
- `#include <textePanneau>` → [60.4. La fonction textePanneau\(\) — écrire dans le panneau](#)
- `#include <nombrePanneau>` → [129.1. La fonction nombrePanneau\(\) — un nombre dans le panneau](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **565 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage – la mise en route de la console
main()         83 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
EcrireTexte    10 o ← textePanneau() – écrit un texte, lettre par lettre
EcrireNombre   33 o ← nombrePanneau() – écrit un nombre en chiffres
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         240 o ← CopierTuiles – 15 lettres sur 43
"SCORE"        5 o ← textePanneau() – un texte gravé
```

Lettres gravées : CEORS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
panneau()      x 1    14 o sur place, dans main()
textePanneau() x 1    11 o sur place, dans main() + EcrireTexte (10 o)
nombrePanneau() x 1   10 o sur place, dans main() + EcrireNombre (33 o)
defiler()      x 1     7 o sur place, dans main()
image()        x 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : le décor glisse sans arrêt, mais le SCORE en haut reste en place.
//
// Le PANNEAU est une deuxième couche, posée par-dessus le fond.
// defiler() fait glisser le fond, mais pas le panneau : parfait pour un score.
//   panneau(x, y)                : place le panneau et l'allume.
//   textePanneau(colonne, ligne, "") : écrit dans le panneau (comme texte()).
//   nombrePanneau(colonne, ligne, n) : écrit un nombre dans le panneau.

#include <panneau>           // montre le panneau, à une place choisie
#include <textePanneau>      // écrit un texte sur le panneau
#include <nombrePanneau>    // écrit un nombre sur le panneau
#include <defiler>          // fait glisser tout le fond
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    uint8_t score = 0;
    uint8_t depuis = 0;     // Les images depuis le dernier point.
    uint8_t glisse = 0;     // De combien de pixels le fond a glissé.

    panneau(0, 0);          // Le panneau, en haut à gauche de l'écran.
    textePanneau(1, 0, "SCORE");

    while (true) {          // Répète pour toujours :
        depuis++;
        if (depuis >= 20) { depuis = 0; score++; nombrePanneau(8, 0, score); }
                                // Toutes les 20 images : un point, et on le réécrit
                                // (seulement quand il change).

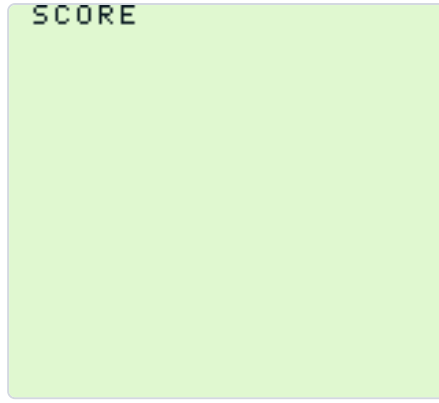
        glisse++;           // Le fond glisse d'un pixel...
        defiler(glisse, 0); // ... mais le panneau ne bouge pas.
        image();            // Attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

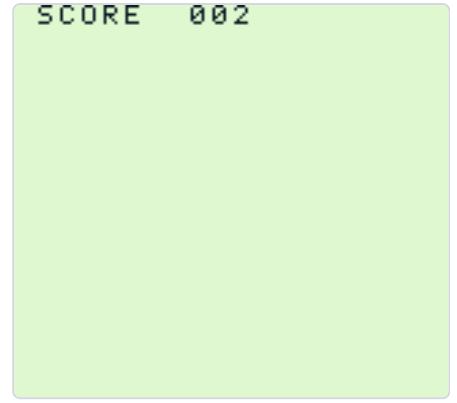
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



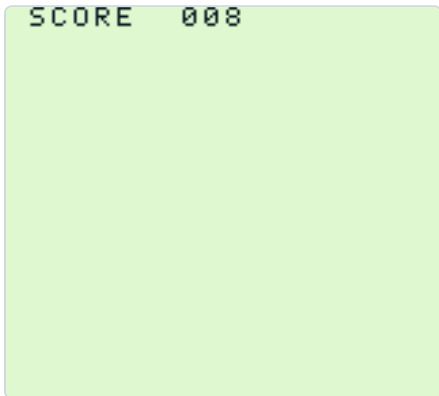
la toute première image — la console vient de démarrer — image 1



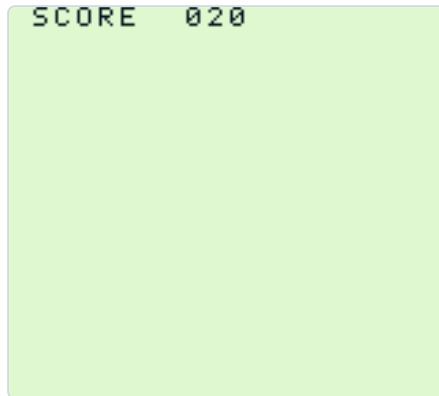
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le score monte dans le panneau pendant que le fond glisse.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le score monte (0 → 6)
- ✓ il ne reste PAS bloqué à zéro
- ✓ le fond défile en même temps
- ✓ et le panneau est allumé

Ensuite : **131. Des ennemis qui vivent et qui meurent** — chapitre 23 : Un vrai jeu. · gameboy3 — écrire une cartouche Game Boy en C++